TECHNICAL ARTICLE & ADMINISTRATION GUIDE

Understanding PostgreSQL Architecture

For Effective Database Administration

Validated against PostgreSQL 18.6 official documentation

Production baseline: PostgreSQL 18 | September 2026

Original article credit "Understanding PostgreSQL Architecture for Effective Database Administration" by Pudi Tirumala Ganesh, published March 4, 2025 on LinkedIn. This document expands, restructures, and validates the subject while preserving full credit to the original article owner.

Document Control

Field	Value
Document type	Technical Article and Administration Guide
Original article author	Pudi Tirumala Ganesh
Original publication	LinkedIn — March 4, 2025
PostgreSQL baseline	18.6
Primary authority	PostgreSQL Global Development Group official documentation
Audience	PostgreSQL DBAs, developers, architects, and operations engineers

Source and Accuracy Statement

The source article supplies the educational sequence: client/server, processes, memory, storage, transactions, MVCC, recovery, and vacuum. PostgreSQL 18 official documentation is the authority for current behavior. The supplied diagram is reproduced in the matching architecture section and is followed by a PostgreSQL 18 accuracy matrix because some labels represent older architecture or mix shared and session-local memory.

Image credit The architecture image was supplied with the source article. Credit remains with the original image creator/owner and the Pudi Tirumala Ganesh LinkedIn article through which it was supplied. Reproduction here is for technical explanation; no ownership is transferred or implied.

Table of Contents / Index

1. Executive Summary

A PostgreSQL instance consists of the main postgres server process, client backend processes, background and auxiliary processes, instance-wide shared memory, per-process/session memory, and persistent database/WAL files. Understanding the boundaries between these components helps administrators size connections and memory, diagnose waits, protect durability, tune vacuum, and design recovery and replication.

Administration question	Architecture evidence
Why are connections expensive?	PostgreSQL normally creates a separate backend process per client session.
Why can one query use more than work_mem?	Several plan nodes and parallel processes can each allocate local memory.
Why can data pages be written after COMMIT?	WAL is flushed first; dirty pages can be written later and recovered by redo.
Why do updates create space pressure?	MVCC keeps old tuple versions until no relevant snapshot needs them.
Why is a standby not a backup?	Physical/logical replication can reproduce unwanted change; recovery needs tested backups/WAL.

2. Client–Server Architecture

Component	Role	Examples
Client/frontend	Opens a session, sends protocol messages/SQL, receives results.	psql, pgAdmin, application drivers, JDBC/ODBC clients.
Main postgres server	Listens, initializes/supervises instance, launches children.	postgres executable; historically called postmaster.
Client backend	Authenticates and executes one client session.	Visible as client backend in pg_stat_activity.
Pooler/proxy	Reuses or routes server connections.	External component; behavior depends on pooling mode.

Component	Role	Examples
Storage/OS	Provides files, page cache, fsync, networking, and process scheduling.	Local/block/network storage subject to durability requirements.

The default TCP port is normally 5432 but is configurable. On supported Unix-like systems, the supervisor commonly forks a process for each connection; Windows uses a different process-creation implementation while preserving process-per-session behavior.

```
SELECT current_database(), current_user, version();
SHOW port;
SHOW listen_addresses;
SHOW max_connections;

SELECT pid, backend_type, usename, datname, client_addr,
     state, backend_start, xact_start, wait_event_type,
     wait_event, left(query,160) AS query
FROM pg_stat_activity
ORDER BY backend_type, backend_start;
```

3. PostgreSQL 18 Architecture Overview

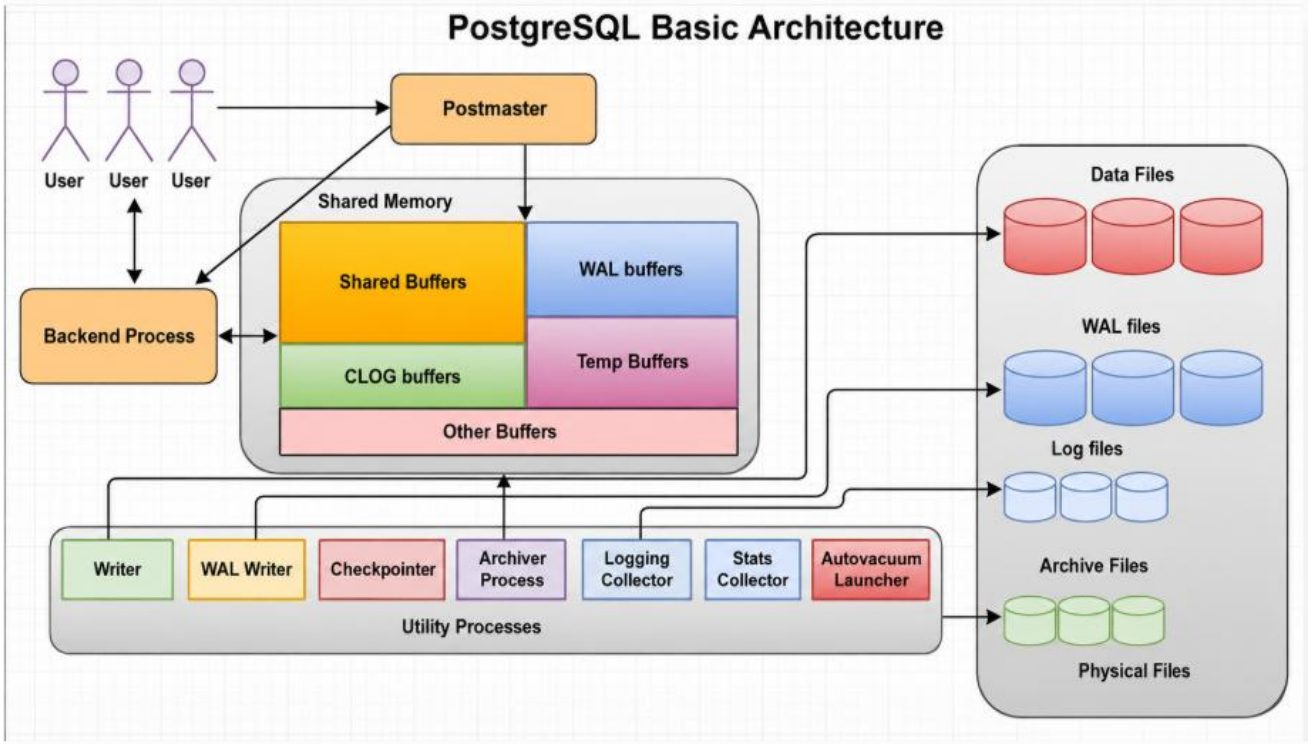

Figure 1 — Supplied "PostgreSQL Basic Architecture" diagram, placed with the source article's architecture overview. Credit remains with the original image owner and source article.

How to read this figure Use it as a high-level teaching diagram, not an exact PostgreSQL 18 process/memory map. The table below identifies labels that require current-version clarification.

Diagram label	PostgreSQL 18 interpretation
Postmaster	Historical name for the supervising postgres server process.
Backend Process	Normally one server process per client connection.
Shared Buffers	Correctly instance-wide shared memory for database pages.
WAL buffers	Correctly shared memory for WAL data not yet written.
Temp Buffers	Session-local buffers for temporary tables—not a general shared-memory pool.
CLOG buffers	Transaction commit-status cache associated with pg_xact; not a user-tuned generic buffer box.
Other Buffers	Too broad; PostgreSQL has many shared structures and process-local memory contexts.
Stats Collector	No dedicated collector process in modern PostgreSQL; cumulative statistics use shared memory and persistence files.
Logging Collector	Optional background process when logging_collector is enabled.
Archive Files	Created by configured archiving; not automatically present in every instance.

4. Server and Backend Processes

The main server establishes shared memory and semaphores, starts auxiliary processes, listens on configured sockets, accepts connection attempts, and supervises child processes. A client backend parses SQL, invokes the rewriter/planner/executor, manages transactions and locks, uses shared and local memory, generates WAL for changes, and communicates directly with its client.

Process/limit	Meaning	DBA concern
client backend	Dedicated process for one connected session.	Idle-in-transaction snapshots and locks; local memory.
max_connections	Maximum regular client connection slots.	Do not size only from demand; include memory and OS capacity.
superuser_reserved_connections	Slots reserved for superusers after normal slots fill.	Maintain emergency administrative access.
reserved_connections	Additional reservation mechanism for roles with pg_use_reserved_connections.	Privilege and capacity governance.
parallel worker	Additional process participating in a parallel plan.	Can multiply CPU and local memory use.
background worker	Built-in or extension process tied to the instance.	Review trust, worker slots, restart/crash behavior.

```
SELECT backend_type, state, count(*)
FROM pg_stat_activity
GROUP BY backend_type, state
ORDER BY backend_type, state;

SELECT count(*) FILTER (WHERE backend_type='client backend') AS clients,
     current_setting('max_connections')::int AS max_connections
FROM pg_stat_activity;
```

5. Background and Auxiliary Processes

Process	PostgreSQL 18 responsibility
background writer	Writes selected dirty shared buffers to create reusable clean buffers.
checkpointer	Performs checkpoints and spreads dirty-page writes across the checkpoint interval.
WAL writer	Writes WAL buffers periodically; committing backends can also flush WAL.

Process	PostgreSQL 18 responsibility
logging collector	Captures stderr and redirects to configured log files when enabled.
autovacuum launcher/workers	Schedules and performs VACUUM/ANALYZE work.
startup process	Replays WAL during crash/archive/standby recovery.
WAL archiver	Archives completed WAL segments when configured.
WAL sender	Streams WAL or logical-decoding output to replication clients.
WAL receiver	Receives streaming WAL on a standby.
WAL summarizer	Creates WAL summaries when summarize_wal is enabled.
logical replication launcher/workers	Coordinates and applies logical replication changes.
parallel workers	Assist eligible query, index-build, and maintenance operations.

Writer correction The background writer does not primarily “reduce checkpoint workload.” It writes selected dirty buffers to maintain clean buffers. The checkpointer remains responsible for ensuring checkpoint requirements, and backends can also write buffers.

```
SELECT pid, backend_type, wait_event_type, wait_event
FROM pg_stat_activity
WHERE backend_type <> 'client backend'
ORDER BY backend_type, pid;

SELECT * FROM pg_stat_checkpointer;
SELECT * FROM pg_stat_wal;
```

6. Memory Architecture

Area	Scope	Purpose
shared_buffers	Instance-wide	Caches table/index pages.
WAL buffers	Instance-wide	Temporarily holds WAL data before writes.

Area	Scope	Purpose
Lock/process/statistics structures	Instance-wide	Coordinates concurrency and shared state.
Dynamic shared memory	Operation-specific shared	Supports parallel operations and extensions.
work_mem	Per eligible operation/process	Sort, hash, memoize and related executor work before spill.
maintenance_work_mem	Per maintenance operation	VACUUM, CREATE INDEX, ALTER TABLE and similar work.
autovacuum_work_mem	Per autovacuum worker	Overrides maintenance memory for autovacuum.
temp_buffers	Per session	Buffers temporary-table pages.
Local memory contexts	Per backend/worker	Parser, planner, executor, catalogs, result processing and extensions.

work_mem correction work_mem is not simply “per query.” A query may have several memory-consuming plan nodes; multiple sessions run at once; parallel workers can each allocate memory. Size it from concurrency and measured plans.

```
SELECT name, setting, unit, context, source, pending_restart
FROM pg_settings
WHERE name IN ('shared_buffers','wal_buffers','work_mem',
 'maintenance_work_mem','autovacuum_work_mem','temp_buffers',
 'max_connections','max_worker_processes','max_parallel_workers',
 'max_parallel_workers_per_gather')
ORDER BY name;
```

7. Query and Transaction Lifecycle

1. Client connects and is authenticated according to pg_hba.conf and role credentials.
2. A backend receives SQL through the frontend/backend protocol.
3. Parser checks syntax and creates a parse tree.
4. Rewriter expands views and applies rules.

5. Planner estimates alternatives using statistics and cost settings.
6. Executor runs the plan, reading shared buffers or performing I/O.
7. Data changes create MVCC tuple versions and WAL records.
8. COMMIT follows WAL durability rules; transaction locks/state are released.
9. Dirty data pages can be written later because WAL supports redo.

```
EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS, VERBOSE)
SELECT * FROM public.orders WHERE customer_id=1001;

SELECT locktype, mode, granted, relation::regclass, pid
FROM pg_locks WHERE pid=pg_backend_pid();
```

8. Storage Architecture

Storage component	Purpose
PGDATA/base	Per-database relation files stored under database OID directories.
global	Cluster-wide catalog relations.
Relation forks	Main data plus free-space map, visibility map, and initialization fork where applicable.
Tables and indexes	Arrays of pages, usually 8 KB; separate relation files.
TOAST	Compresses/stores oversized attribute values out of line.
pg_wal	WAL segment files for recovery and replication.
pg_xact	Transaction commit-status data; historically called CLOG.
pg_multixact	Multitransaction status used for shared row locks.
pg_tblspc	Symbolic links to tablespace locations.
pg_control	Critical cluster control and checkpoint state.

```
SELECT c.oid::regclass AS relation,
    pg_size_pretty(pg_relation_size(c.oid)) AS main_fork,
    pg_size_pretty(pg_indexes_size(c.oid)) AS indexes,
    pg_size_pretty(pg_total_relation_size(c.oid)) AS total,
    t.spcname AS tablespace
FROM pg_class c
LEFT JOIN pg_tablespace t ON t.oid=c.reltablespace
```

```
WHERE c.relkind IN ('r','m')
ORDER BY pg_total_relation_size(c.oid) DESC LIMIT 30;
```

Tablespace warning A tablespace is a filesystem location referenced by PostgreSQL, not an independent backup/recovery unit. File-level copying of selected relation files is not a valid logical table backup.

9. WAL, Checkpoints, and Crash Recovery

WAL requires change records to reach durable storage before corresponding dirty data pages can be safely written. At checkpoint, data files are flushed through a defined point and a checkpoint record/control state is maintained. After a crash, the startup process reads control/checkpoint information and replays WAL from the redo location to restore consistent data files.

Stage	Action
Change	Backend modifies a buffer and inserts WAL records.
Commit	Required WAL is flushed according to synchronous_commit and durability settings.
Page write	Dirty heap/index page can be written after its WAL is safely flushed.
Checkpoint	Dirty pages are flushed; redo point advances.
Crash recovery	Startup process replays WAL forward; incomplete transactions remain invisible under MVCC.
Archive/PITR	Base backup plus continuous WAL archive supports recovery to a target.

```
SELECT pg_current_wal_lsn(), pg_current_wal_insert_lsn();
SELECT * FROM pg_stat_wal;
SELECT * FROM pg_stat_checkpointer;
SHOW checkpoint_timeout;
SHOW max_wal_size;
SHOW full_page_writes;
SHOW synchronous_commit;
```

10. Transactions, ACID, and MVCC

Property/concept	PostgreSQL meaning
Atomicity	Transaction effects commit together or remain uncommitted/are rolled back.
Consistency	Constraints and correct application logic preserve valid state; ACID does not create business rules automatically.
Isolation	Behavior follows Read Committed, Repeatable Read, or Serializable semantics.
Durability	Committed effects survive according to WAL/fsync/synchronous_commit design.
MVCC snapshot	Statements/transactions see tuple versions allowed by the isolation snapshot.
Row/table/advisory locks	Still exist for conflicts and explicit coordination; MVCC does not remove all blocking.
Serializable SSI	Detects dangerous dependency patterns and may abort a transaction for retry.

Snapshot correction PostgreSQL uses MVCC to provide consistent snapshots, but it does not mean every transaction uses snapshot isolation. Read Committed obtains a new snapshot per command; Repeatable Read uses one transaction snapshot; Serializable adds SSI checks.

```
SHOW default_transaction_isolation;
SELECT txid_current_if_assigned(), pg_current_snapshot();

SELECT pid, state, xact_start, now()-xact_start AS xact_age,
     age(backend_xmin) AS xmin_age, left(query,160) AS query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY xact_start;
```

11. Vacuum and Autovacuum

UPDATE and DELETE leave obsolete tuple versions until they are no longer visible. Standard VACUUM removes dead tuple/index references where safe, makes space reusable, updates visibility/free-space

information, and freezes old transaction metadata. ANALYZE refreshes planner statistics. Autovacuum normally performs routine VACUUM and ANALYZE dynamically.

Operation	Lock/space behavior
VACUUM	Normal DML continues; substantial I/O possible; space usually remains for table reuse.
VACUUM ANALYZE	Adds planner-statistics refresh.
VACUUM FULL	ACCESS EXCLUSIVE rewrite; extra disk; returns excess space to OS.
Autovacuum	Launcher plus workers; threshold/insert/freeze driven; never runs VACUUM FULL.
ANALYZE parent partitioned table	May be required manually because parent stores no tuples and is not autoanalyzed.

```
SELECT schemaname, relname, n_live_tup, n_dead_tup,
    last_autovacuum, last_autoanalyze,
    autovacuum_count, autoanalyze_count
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC LIMIT 30;

SELECT pid, relid::regclass AS relation, phase,
    heap_blks_scanned, heap_blks_total, index_vacuum_count
FROM pg_stat_progress_vacuum;
```

12. Replication and High Availability

Component	Role
WAL sender	Streams physical WAL or logical decoding output from upstream.
WAL receiver	Receives physical WAL on standby.
Startup process	Replays WAL on standby/recovery server.
Replication slot	Retains required WAL and sometimes tuple/catalog horizons for a consumer.
Synchronous replication	Can require configured standby confirmation before commit completes.

Component	Role
Logical replication workers	Copy/apply published changes at table level.
Archive recovery	Restores archived WAL after a base backup for PITR/standby use.

```
SELECT application_name, client_addr, state, sync_state,
    sent_lsn, write_lsn, flush_lsn, replay_lsn
FROM pg_stat_replication;

SELECT slot_name, slot_type, active, restart_lsn,
    confirmed_flush_lsn, wal_status, safe_wal_size
FROM pg_replication_slots;

SELECT status, written_lsn, flushed_lsn, latest_end_lsn
FROM pg_stat_wal_receiver;
```

HA/backup distinction Replication improves availability but is not a backup. Maintain tested base/logical backups, WAL archives as required, restore/PITR procedures, and defined RPO/RTO.

13. Administration and Monitoring Scripts

13.1 Blocking and waits

```
SELECT pid, usename, state, wait_event_type, wait_event,
    pg_blocking_pids(pid) AS blockers, left(query,180) AS query
FROM pg_stat_activity
WHERE cardinality(pg_blocking_pids(pid)) > 0;

SELECT wait_event_type, wait_event, count(*)
FROM pg_stat_activity
WHERE wait_event IS NOT NULL
GROUP BY wait_event_type, wait_event ORDER BY count(*) DESC;
```

13.2 Database and relation health

```
SELECT datname, numbackends, xact_commit, xact_rollback,
    blks_read, blks_hit, temp_files, temp_bytes, deadlocks
FROM pg_stat_database
WHERE datname IS NOT NULL ORDER BY datname;
```

```
SELECT schemaname, relname, seq_scan, idx_scan,
    n_live_tup, n_dead_tup, last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC LIMIT 30;
```

13.3 PostgreSQL 18 I/O view

```
SELECT backend_type, object, context,
    reads, read_bytes, read_time,
    writes, write_bytes, write_time,
    extends, extend_bytes, fsyncs, fsync_time
FROM pg_stat_io
ORDER BY backend_type, object, context;
```

13.4 Configuration provenance

```
SELECT name, setting, unit, context, source,
    sourcefile, sourceline, pending_restart
FROM pg_settings
WHERE name IN ('max_connections','shared_buffers','work_mem',
 'maintenance_work_mem','max_worker_processes','max_wal_senders',
 'checkpoint_timeout','max_wal_size','autovacuum_max_workers',
 'logging_collector','archive_mode')
ORDER BY name;
```

14. Troubleshooting Scenarios

Symptom	Architecture evidence	Direction
Connection saturation	Backend counts/states, pool, max/reserved slots, memory.	Fix leaks/idle transactions; pool; size limits safely.
OOM/high memory	Concurrent backends, plan nodes, parallel workers, maintenance.	Reduce concurrency/local memory; review pooling and OS logs.
Slow query	EXPLAIN buffers/WAL, waits, statistics, temp spills, I/O.	Tune query/index/stats/memory after evidence.
Checkpoint spikes	pg_stat_checkpointer, WAL rate, pg_stat_io, storage latency.	Spread checkpoints; review max_wal_size/timeout and storage.
WAL growth	Slots, archiver, lag, backup, wal_keep_size.	Restore consumer/archive; remove only verified abandoned slot.

Symptom	Architecture evidence	Direction
Autovacuum lag	Dead tuples, progress, old xmin, workers, locks, I/O.	Resolve confirmed blockers; tune per table/capacity.
Replica lag	Sender/receiver/replay LSN, network/storage, conflicts.	Locate send/write/flush/replay bottleneck.
Server start failure	Logs, port, permissions, disk, config, postmaster.pid context.	Resolve exact error; do not delete files blindly.
Crash recovery slow	WAL distance, I/O, archive restore throughput.	Allow/monitor recovery; tune future checkpoint/archive design.
Logging absent	logging_collector, log_destination, permissions, service logs.	Validate actual logging route and rotation.
Tablespace unavailable	Mount/path/permissions and pg_tblspc link.	Restore supported storage; do not relink casually.
MVCC bloat	Long snapshots, slots, update churn, vacuum history.	Fix horizon/cadence; targeted standard VACUUM.

15. Production Design Guidance

Area	Guidance
Connections	Use a pooler where appropriate; retain administrative access; avoid oversized max_connections.
Memory	Budget instance shared memory plus worst-case local memory at peak concurrency/parallelism.
Storage	Use durable storage honoring fsync; maintain headroom for pg_wal, temp, maintenance, and backups.
Checkpoints	Monitor cause, write/sync time, WAL rate, and device latency; balance recovery time and I/O.

Area	Guidance
Vacuum	Keep autovacuum enabled; tune large/hot tables per workload; watch XID/MultiXact age.
Logging	Protect sensitive values; centralize and rotate logs; enable useful checkpoint/autovacuum evidence.
Security	Least privilege, restrictive pg_hba.conf, SCRAM/TLS as required, and protected files.
HA/DR	Document RPO/RTO; test failover, backup restore, WAL archive, and PITR.
Extensions	Review compatibility, shared_preload_libraries, background workers, privileges, and upgrade path.
Change control	Test on production-like data; record owner, rollback, metrics, and post-change validation.

16. Corrections to the Source Notes

Source statement/diagram	PostgreSQL 18 validated correction
Postmaster is the product/server name.	The server program/process is postgres; postmaster is the historical supervisor term.
Background writer writes dirty buffers to disk and reduces checkpoints.	It writes selected dirty shared buffers to create clean buffers; checkpointer still satisfies checkpoints.
Checkpointer “ensures data consistency.”	WAL rules, checksums/storage behavior, checkpoints, and recovery work together; a checkpoint is not the sole integrity mechanism.
WAL writer ensures durability.	WAL writer helps write WAL periodically; commit backends and configured flush rules are central to commit durability.
Autovacuum reclaims storage.	Standard vacuum mostly makes internal space reusable; it does not normally shrink files and also freezes tuples/analyzes.

Source statement/diagram	PostgreSQL 18 validated correction
Logger always writes PostgreSQL files.	Logging collector is optional; logs may go to stderr, csvlog/jsonlog, syslog, eventlog, or service infrastructure.
Shared buffers cache “frequently accessed” blocks.	They cache pages brought into PostgreSQL buffers; replacement/use is workload driven, not a guaranteed frequency tier.
work_mem is per query.	It is a base limit per eligible operation/process, potentially multiple times per query.
PostgreSQL uses MVCC to provide snapshot isolation.	MVCC provides snapshots, but isolation semantics vary; Read Committed is not transaction-wide snapshot isolation.
Vacuum prevents bloat.	It controls dead-tuple accumulation and promotes reuse; physical bloat can remain and root causes matter.
Stats Collector is a dedicated process.	Modern PostgreSQL cumulative statistics use shared memory; the old collector-process diagram is outdated.
Temp buffers are in shared memory.	temp_buffers is session-local and only for temporary tables.
CLOG is a generic user-tunable buffer.	Commit status is stored in pg_xact and cached internally; CLOG is historical terminology.
Archive files always exist.	They exist only when WAL archiving is configured and working.

17. Production Checklist

- ☐ Exact PostgreSQL 18.x version, OS, package/service, storage, and topology recorded.
- ☐ Client/backend counts, pool behavior, transaction age, waits, and reserved access monitored.
- ☐ Shared and local memory budget tested at realistic concurrency and parallelism.
- ☐ WAL generation, checkpoints, pg_wal space, archive status, and storage latency alerted.
- ☐ Autovacuum progress, dead tuples, XID/MultiXact age, and blockers monitored.
- ☐ Replication sender/receiver/replay lag and slot retention monitored.
- ☐ Backups, restore, PITR, crash recovery, and failover tested.
- ☐ pg_hba.conf, TLS/SCRAM, roles, logging exposure, and filesystem ownership reviewed.

- ☐ Tablespace dependencies and mount recovery are documented.
- ☐ Architecture diagrams are treated as teaching aids; live PG18 views/configuration are operational truth.
- ☐ Every change has nonproduction test, owner, rollback, and post-change evidence.

18. Credits and Official References

18.1 Credits

Item	Credit
Original article	Pudi Tirumala Ganesh — "Understanding PostgreSQL Architecture for Effective Database Administration," March 4, 2025.
Original article URL	https://www.linkedin.com/pulse/understanding-postgresql-architecture-effective-pudi-tirumala-ganesh-ibxvf/
Author profile	https://www.linkedin.com/in/ptganesh/
Supplied diagram	Credit remains with its original creator/owner and the source article through which it was supplied.
Technical adaptation	Prepared for Praveen Madupu and validated against PostgreSQL 18.6 official documentation.

18.2 PostgreSQL official references

- **Architectural Fundamentals:** https://www.postgresql.org/docs/18/tutorial-arch.html
- **postgres Server Program:** https://www.postgresql.org/docs/18/app-postgres.html
- **Server Startup:** https://www.postgresql.org/docs/18/server-start.html
- **Resource Consumption and Memory:** https://www.postgresql.org/docs/18/runtime-config-resource.html
- **Database File Layout:** https://www.postgresql.org/docs/18/storage-file-layout.html
- **Database Page Layout:** https://www.postgresql.org/docs/18/storage-page-layout.html
- **Tablespaces:** https://www.postgresql.org/docs/18/manage-ag-tablespaces.html
- **WAL Introduction:** https://www.postgresql.org/docs/18/wal-intro.html
- **WAL Configuration:** https://www.postgresql.org/docs/18/wal-configuration.html
- **WAL Internals:** https://www.postgresql.org/docs/18/wal-internals.html

- **MVCC Introduction:** https://www.postgresql.org/docs/18/mvcc-intro.html
- **Transaction Isolation:** https://www.postgresql.org/docs/18/transaction-iso.html
- **Routine Vacuuming:** https://www.postgresql.org/docs/18/routine-vacuuming.html
- **Monitoring Statistics:** https://www.postgresql.org/docs/18/monitoring-stats.html
- **Error Reporting and Logging:** https://www.postgresql.org/docs/18/runtime-config-logging.html
- **Streaming Replication:** https://www.postgresql.org/docs/18/warm-standby.html
- **Logical Replication Architecture:** https://www.postgresql.org/docs/18/logical-replication-architecture.html
- **Background Worker Processes:** https://www.postgresql.org/docs/18/bgworker.html